

From No-Code Heuristics to Explicit Gradient Control: A Controlled Study of Hyperparameter Tuning and Loss-Curve Diagnosis in a Keras Intent Classifier

Carl Angelo Patano
College of Computer Studies
Jose Rizal University
Mandaluyong City, Philippines
carlangelo.patano@my.jru.edu
ITC-C508

Abstract—This paper reports a controlled study of a Keras feed-forward text classifier trained to sort telecommunications support messages into three intents, and compares it against a no-code baseline built on the same task in a prior exercise. Seven training runs were executed under a fixed random seed, varying dataset size, hidden dimension, learning rate, dropout, weight decay, and training length. Dataset size, not architecture, produced the decisive effect: holding every hyperparameter constant while expanding the corpus from 40 to 225 examples moved validation macro F1 from 0.0952 to 0.9388 and ROC-AUC from 0.3333 to 0.9987. Four subsequent configurations produced identical predictions, indicating that tuning had saturated. The champion exceeded the no-code benchmark on a random validation split, 0.9388 against 0.9153, but fell below it on the eleven adversarial phrases the baseline was originally scored against, 0.8222. The choice of evaluation set, not the model, determined which system appeared superior.

Keywords—neural networks, hyperparameter tuning, loss curve diagnosis, overfitting, regularization, reproducibility, benchmark comparison, text classification

I. INTRODUCTION

Exercise PT-P1 established a three-class intent classifier for telecommunications subscriber messages using Machine Learning for Kids, a no-code supervised learning platform [1]. That study concluded that dataset representativeness mattered more than dataset volume: fifteen additional ordinary sentences raised accuracy by nine percentage points and corrected none of four linguistic failures, while ten failure-targeted sentences raised it by a further eighteen and corrected three of them.

The platform, however, exposed no control over the learning process itself. Its internals were inaccessible, its

training procedure was a progress bar, and the only lever available to the operator was the training data. The present exercise removes that constraint. The same classification task is reimplemented in TensorFlow/Keras [2], where learning rate, capacity, regularization, optimizer, and training duration are explicit and adjustable, and where the loss at every epoch is observable.

Two questions follow. The first is whether explicit control produces a better classifier than automated heuristics. The second, which proved the more interesting of the two, is whether that comparison can be stated at all without first specifying the test set on which it was measured.

The benchmark to be exceeded is the final PT-P1 model, which scored 90.91 percent accuracy and a macro F1 of 0.9153 on eleven held-out phrases. One detail of that figure is not cosmetic. All PT-P1 metrics are macro averages, weighting each class equally regardless of size, whereas the provided Keras notebook computed a weighted F1. A weighted score cannot be compared against a macro one, so the notebook was extended to report both, and every comparison in this paper is macro against macro.

The paper is organised as follows. Section II describes the corpora, the architecture, the reproducibility procedure, and the experimental design. Section III reports all seven runs with their loss curves and diagnoses. Section IV discusses the debrief questions on code-based versus no-code optimization, loss-curve diagnosis, and the relationship between data volume and architectural capacity. Section V addresses, in a dedicated treatment, where each stage of the training loop occurs in the implemented pipeline. Section VI states the limitations that remain, and Section VII concludes.

II. METHODOLOGY

A. Corpora and Partitioning

Two corpora were used. The original corpus is the 40-example PT-P1 training set, class-imbalanced at fifteen Negative_Complaint, twelve Positive_Praise, and thirteen Urgent_Support. The expanded corpus contains 225 examples balanced at seventy-five per class, constructed by adding paraphrases within the same domain and deliberately extending coverage of negation, sarcasm, and time-pressure phrasing, which were the three constructions PT-P1 identified as failure modes.

Both corpora were partitioned into seventy percent training, fifteen percent validation, and fifteen percent test, stratified on the label at both split stages. Stratification is necessary at this scale because without it a class can vanish entirely from the validation set, leaving per-class F1 undefined. The test partition was held out completely until the champion configuration had been selected on validation loss.

TABLE I. CORPUS PARTITIONS

Corpus	Total	Train	Val.	Test	Vocab.
Original	40	28	6	6	233
Expanded	225	157	34	34	562

Split ratio 70 / 15 / 15, stratified on the label at both stages.

B. Architecture and Preprocessing

The pipeline is an embedding layer, followed by global average pooling across the token axis, a dense hidden layer with ReLU activation, a dropout layer, and a softmax output layer. Sentences are lowercased, whitespace-tokenised, and mapped to integer indices against a vocabulary capped at one thousand tokens, with index zero reserved for padding and index one for unknown words. Every sequence is padded or truncated to twenty positions; the longest sentence in either corpus is fourteen words, so no truncation occurs in practice.

The output layer size is not a design choice. Three classes mandate three softmax neurons, and the notebook reads this from the data rather than hardcoding it, so the layer remains correct if a fourth class is ever introduced.

C. Reproducibility

The provided notebook sets the NumPy and TensorFlow global seeds to forty-two. This was tested and found insufficient. EXP-01 was executed, then re-executed under an identical configuration in a clean session, and the final training loss moved from 0.5933 to approximately 0.55 while the shape of the curve was preserved.

Three sources of variation survive those two lines. Python’s own random module is never seeded. TensorFlow derives each operation’s seed from an internal session counter, so the initial weights depend on how many operations were created earlier in that session, and a clean restart therefore diverges from a session with prior cell executions. Finally, GPU and some CPU kernels use non-deterministic reduction orders.

Seeding Python’s random module, calling the combined Keras seed utility, and enabling operation determinism eliminated the drift. Verification was performed twice: two consecutive clean-session runs of EXP-01 produced identical values to four decimal places, and EXP-01B reproduced exactly when re-run after six intervening experiments.

This matters beyond housekeeping. An experiment log records a seed so that a configuration can be re-run and checked. Where the seed does not determine the result, every logged row is unverifiable, and the difference between two runs reflects initialisation noise as much as the hyperparameter that was deliberately changed. All runs were executed on Google Colab free tier, CPU runtime, TensorFlow 2.20.0, with hardware held constant for the reason given above.

D. Experimental Design

Seven configurations were executed. EXP-01 reproduces the provided baseline on the original corpus. EXP-01B repeats it on the expanded corpus with every other setting unchanged. EXP-02 and EXP-03 were configured to induce underfitting and overfitting respectively. EXP-04 applies regularisation to high capacity. EXP-05 extends the training budget with early stopping, and EXP-05B adds weight decay.

TABLE II. HYPERPARAMETER CONFIGURATIONS

Run	Corpus	Hid.	LR	Drop.	L2	Ep.
EXP-01	orig.	64	0.005	0.1	—	15
EXP-01B	exp.	64	0.005	0.1	—	15
EXP-02	exp.	16	0.0001	0.0	—	15
EXP-03	exp.	256	0.05	0.0	—	15
EXP-04	exp.	128	0.005	0.4	—	15
EXP-05	exp.	64	0.005	0.1	—	60
EXP-05B	exp.	64	0.005	0.1	1e-4	60

Embedding dimension 32, batch size 32, Adam optimizer, and seed 42 throughout. EXP-05 and EXP-05B use early stopping on validation loss with patience 8 and best-weight restoration.

Two design properties must be stated plainly. EXP-01B is the only run that differs from EXP-01 in exactly one variable, the corpus, with every hyperparameter, the seed, and the architecture held identical; it therefore supports the strongest causal claim in the series. EXP-02, by contrast, varies three parameters simultaneously as prescribed by the exercise brief. Its purpose is to induce a condition for diagnostic practice rather than to attribute a cause, and it is reported as such. EXP-05B, which differs from EXP-05 only in weight decay, is the second clean single-variable comparison.

E. Extensions to the Provided Code

Three techniques absent from the provided notebook were implemented as configuration flags defaulting to off, so that EXP-01 reproduces the baseline exactly and any later effect remains attributable. Early stopping monitors validation loss with best-weight restoration, which matters

because without it the model retains the final weights rather than the best ones. Weight decay applies an L2 penalty to the hidden layer. ROC-AUC is computed one-versus-rest from predicted probabilities rather than thresholded labels, since the measure concerns ranking quality across all thresholds and hardened labels discard exactly that information.

III. RESULTS

Table III reports every run. Diagnoses follow from the loss curves in Figs. 1 through 6 and from the numerical gap between the final training and validation losses.

TABLE III. RESULTS ACROSS ALL SEVEN RUNS

Run	Train	Val.	Gap	Acc.	F1	AUC	Best
EXP-01	0.6305	1.1555	0.5250	16.67	0.0952	0.3333	1/15
EXP-01B	0.0019	0.1200	0.1181	94.12	0.9388	0.9987	15/15
EXP-02	1.0842	1.0888	0.0046	55.88	0.5058	0.8268	15/15
EXP-03	0.0000	1.0842	1.0842	94.12	0.9410	0.9875	2/15
EXP-04	0.0014	0.1387	0.1373	94.12	0.9388	0.9987	13/15
EXP-05	0.0001	0.1067	0.1066	94.12	0.9388	0.9987	48/56
EXP-05B	0.0074	0.1142	0.1068	94.12	0.9388	0.9987	48/56

Train and Val. are final-epoch losses; Gap is their difference. Acc. is validation accuracy in percent, F1 is validation macro F1, AUC is weighted one-versus-rest ROC-AUC. Best is the epoch of lowest validation loss over epochs run. EXP-05 is the champion.

A. EXP-01: The Provided Baseline Fails Below Chance

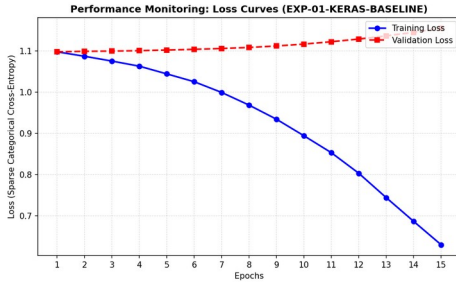


Fig. 1. Provided baseline on 40 examples. Validation loss rises from the first epoch while training loss falls; the best epoch is 1 of 15.

Validation loss rises from 1.0982 to 1.1555 while training loss falls to 0.6305 and training accuracy reaches one hundred percent. The best epoch is the first, meaning the model never generalised better than it did at initialisation.

Validation accuracy of 16.67 percent is below the 33.3 percent chance level for three classes, and a ROC-AUC of 0.3333 confirms that the ranking is worse than random. The confusion matrix explains why: five of the six validation samples were assigned to Negative_Complaint. The model did not learn three classes; it collapsed to majority-class prediction and scored one correct by coincidence.

A structural detail compounds this. With twenty-eight training samples and a batch size of thirty-two, the batch exceeds the dataset, producing a single weight update per epoch and fifteen updates in total, against the seventy-five

available on the expanded corpus. The provided defaults are not merely suboptimal at this scale; the batch size silently disables most of the training.

B. EXP-01B: Dataset Size in Isolation

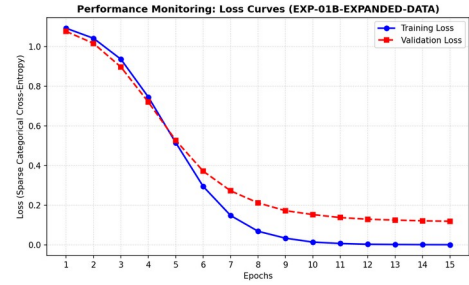


Fig. 2. Identical configuration on 225 examples. Validation loss falls monotonically to 0.1200. Corpus size is the only variable changed from Fig. 1.

With every hyperparameter, the seed, and the architecture held constant, expanding the corpus from forty to 225 examples moves validation loss from 1.1555 to 0.1200 and macro F1 from 0.0952 to 0.9388. Validation loss falls at every epoch; in EXP-01 it rose at every epoch.

TABLE IV. VALIDATION CONFUSION MATRICES, EXP-01 AND EXP-01B

Actual	Neg	Pos	Urg	Neg	Pos	Urg
Negative	1	1	0	11	0	0
Positive	2	0	0	1	9	1
Urgent	2	0	0	0	0	12

Left block: EXP-01, 40 examples. Right block: EXP-01B, 225 examples. Rows are actual classes, columns predicted. Configuration and seed are identical across both.

Placed side by side, these are the clearest results in the study. The left block shows total class collapse; the right shows two classes classified perfectly and the third at nine of eleven. Validation accuracy in EXP-01B had already reached 91.18 percent by the second epoch and improved only to 94.12 percent across the remaining thirteen, so nearly all learning occurred immediately.

C. EXP-02: Underfitting, and the First Metric Disagreement

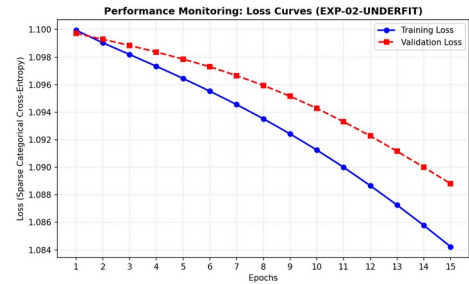


Fig. 3. Underfitting. Note the vertical axis: the entire plotted range spans 0.016 of loss, all of it just below $\ln(3) = 1.0986$, the chance-level cross-entropy for three classes. The apparent descent is an artefact of the scale.

Both curves sit at 1.0842 and 1.0888 against a chance-level cross-entropy of 1.0986, so the model moved roughly one percent off its random initialisation across seventy-five weight updates. Fig. 3 illustrates a second diagnostic hazard alongside the first. The curves appear to descend steadily and in parallel, which on a casual reading resembles healthy convergence, but the vertical axis spans only 0.016 of loss in total. Loss curves must be read against their absolute scale and not their apparent slope.

The instructive result is a contradiction between two metrics. Training accuracy reads 71.34 percent at a chance-level loss. The softmax outputs are near-uniform, so the argmax operation selects correctly often enough to score seventy-one percent while the model carries almost no confidence and has functionally learned nothing. Accuracy reported competence; loss reported chance; loss was correct. A ROC-AUC of 0.8268 adds nuance, indicating that the embedding did learn separable structure and that the classifier never received a step size large enough to act on it.

D. EXP-03: Overfitting, and the Second Metric Disagreement

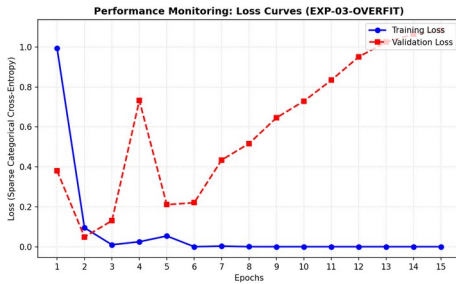


Fig. 4. Overfitting at an excessive learning rate. Validation loss reaches its minimum of roughly 0.05 at the second epoch, oscillates violently through the sixth, then climbs monotonically to 1.0842. Training loss is itself non-monotonic between epochs three and five.

Training loss reaches exactly zero while validation loss stands at 1.0842, a gap of 1.0842 against EXP-01B’s 0.1181. Yet this run holds the highest macro F1 in the entire series at 0.9410, marginally above EXP-01B and EXP-04 at 0.9388.

The curve in Fig. 4 is more informative than either figure. Validation loss reaches approximately 0.05 at the second epoch, which is lower than any value recorded anywhere in this study, including the champion’s 0.1067. It then oscillates sharply, rising to roughly 0.73 at the fourth epoch, falling back to 0.21 at the fifth, and climbing monotonically from the seventh onward. Training loss is itself non-monotonic across epochs three to five, rising after having nearly reached zero.

That oscillation is the documented signature of an excessive learning rate. Each update overshoots the minimum, the correction overshoots in the opposite direction, and the amplitude compounds. The run is therefore two failures at once: a learning rate too large by an order of

magnitude, and the absence of any mechanism to stop training at the point where the model was performing best.

The practical consequence is worth stating directly. Had early stopping been enabled, this configuration would have halted near the second epoch and become the champion of the series. Its poor final numbers record not an incapable model but a training procedure allowed to continue for thirteen epochs past its own optimum. The best-epoch column is what exposes this; the final-epoch metrics conceal it entirely.

The explanation is calibration [8]. A training loss of zero means the model emits probabilities of essentially one for its selected class. When it is correct on validation this costs almost nothing, but when it is wrong, cross-entropy penalises a confident error severely, and two maximally confident errors suffice to drive validation loss to 1.0842 while the accuracy count remains high. ROC-AUC declined from 0.9987 to 0.9875, indicating that ranking quality degraded even as accuracy nominally improved.

EXP-03 was therefore rejected as champion despite holding the highest F1. A model with zero training loss and a gap of 1.0842 has memorised rather than generalised and will fail catastrophically rather than gracefully on input outside its training distribution. The metric chosen for selection determines the model selected.

E. EXP-04: Regularisation Buys Calibration, Not Correctness

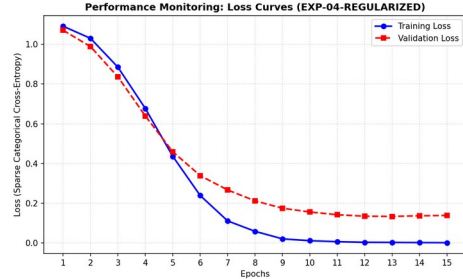


Fig. 5. Dropout at 0.4 narrows the gap eightfold and delays the overfitting turn from epoch 2 to epoch 13, without altering any prediction.

Raising dropout to 0.4 at 128 hidden units reduced the gap from 1.0842 to 0.1373, restored ROC-AUC to 0.9987, and moved the best epoch from the second to the thirteenth, delaying the onset of overfitting by eleven epochs [4].

Accuracy and macro F1 did not change at all, and the confusion matrix is identical to EXP-01B’s in every cell. Dropout at four times the baseline rate, with double the capacity, produced precisely the same thirty-two of thirty-four predictions. Regularisation did not alter which answers the model gave; it altered how justified the confidence behind them was. Only validation loss and ROC-AUC detected the improvement, and accuracy and F1 were blind to it.

F. EXP-05 and EXP-05B: Extended Training and a Null Result

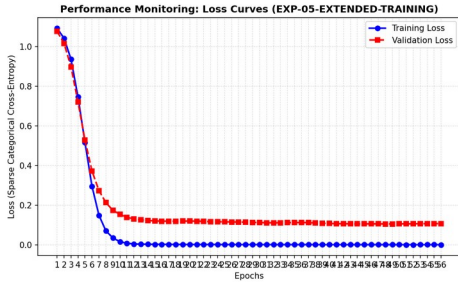


Fig. 6. Champion. Validation loss flattens by roughly the twelfth epoch and drifts between 0.10 and 0.12 for the remaining forty-four. Early stopping halts at epoch 56 and restores weights from epoch 48, within that plateau.

EXP-01B’s best epoch was the fifteenth of fifteen, meaning validation loss was still improving when the run ended, which suggested the training budget had been cut short. Given a sixty-epoch budget with patience eight, training halted at epoch fifty-six and restored weights from epoch forty-eight, reaching a validation loss of 0.1067, the lowest recorded among the well-fitted runs.

Fig. 6 qualifies that result. Validation loss flattens by approximately the twelfth epoch and then drifts between 0.10 and 0.12 for the remaining forty-four, and epoch forty-eight sits inside that plateau rather than at a distinct minimum. The improvement of 0.0133 over EXP-01B is therefore better described as the lower edge of the plateau’s own variation than as a superior optimum located by longer training. Accuracy, F1, ROC-AUC, and the confusion matrix are unchanged. The finding strengthens rather than weakens the saturation argument developed below: even the champion’s margin over the fifteen-epoch run is of uncertain significance.

EXP-05B differs from EXP-05 only in weight decay. The L2 penalty raised training loss by a factor of seventy-four, from 0.0001 to 0.0074, confirming that it was active, while validation loss slightly worsened to 0.1142 and every prediction remained identical. This is reported as a null result rather than omitted. It is the only run in the series that isolates a single variable against its immediate predecessor, and it demonstrates that a penalty applied to a model not suffering from oversized weights costs fit and returns nothing.

G. Saturation

Four configurations, EXP-01B, EXP-04, EXP-05, and EXP-05B, produced identical validation confusion matrices despite differing in capacity, dropout rate, weight decay, and training length. At 157 training examples the model extracts what the validation set can measure, and hyperparameter tuning has saturated against it. Further gains require data rather than configuration.

The word validation carries weight in that sentence. EXP-01B and the champion agree on every validation prediction, yet they differ on the held-out test split, where EXP-01B scored 100 percent and the champion 97.06 percent. Saturation therefore does not establish that the configurations are equivalent; it establishes that a thirty-four-sample

validation set cannot distinguish between them. The champion was nonetheless selected correctly, on validation loss, before any test partition was consulted.

One qualification follows from Fig. 4. EXP-03 reached a lower validation loss at its second epoch than any configuration reached at any point, so the saturation observed here is a property of the training procedures used rather than a hard ceiling on the architecture. What the series demonstrates is that the settings tested converge to the same predictions, not that no setting could do better.

IV. DISCUSSION

A. Code-Based Optimization Versus No-Code Heuristics

The first debrief question asks how explicit manipulation of loss functions, learning rates, and gradient updates differs from reliance on automated heuristics, and what trade-offs each carries.

The no-code platform offered one lever, the training data. Every failure had to be addressed by adding examples, and PT-P1’s central finding, that targeted examples outperform additional volume, was in part a consequence of having no other instrument available.

Explicit control changed the work in three ways. Diagnosis became possible: the platform reported a confidence percentage and nothing else, so a model that had underfit was indistinguishable from one that had overfit, since both simply produced wrong answers. The per-epoch loss curve makes that distinction immediate, and EXP-02 against EXP-03 shows two failures that are indistinguishable by accuracy yet unmistakable on the curves. Failure could also be induced deliberately, which is how EXP-02 and EXP-03 were configured, and no such instrument exists in a no-code environment.

Control did not, however, confer improvement. The decisive variable in this study was again the dataset, and hyperparameter tuning across five configurations on the expanded corpus produced no change in any prediction. The advantage of explicit control lay in understanding the model rather than in improving it.

The trade-off is cost. The no-code baseline required no reproducibility engineering, whereas the Keras implementation required discovering and correcting a seeding defect before any number could be trusted. It required choosing a loss function, an averaging convention, and a batch size, and the provided batch size silently reduced EXP-01 to fifteen weight updates. Automation removes those failure modes precisely by removing the choices.

B. Diagnosing Model Fit and the Role of Dropout and Capacity

The second debrief question asks which behaviours in the training and validation curves indicated overfitting as against

underfitting, and how dropout and hidden dimension resolved or exacerbated each.

The signature of underfitting, seen in EXP-02, is both curves flat, close together, and high, near the chance-level cross-entropy. The gap is 0.0046, which is the diagnostic trap: a small gap indicates a good fit only when the absolute level is also low. EXP-02’s gap is smaller than the champion’s 0.1066, yet EXP-02 is the worse model by every other measure. Gap and level must be read together.

The signature of overfitting, seen in EXP-03, is training loss collapsing toward zero while validation loss turns upward and the gap widens. The numerically decisive evidence is not the shape of the curve but the best epoch: two of fifteen means the optimum was passed after two epochs and the remaining thirteen degraded the model. A third signature is present in the same figure. The violent oscillation of validation loss between the third and sixth epochs, rising to 0.73 and falling back to 0.21, is the compounding overshoot produced by an excessive learning rate, and it is visible on the curve well before the final loss value reveals anything.

Raising the hidden dimension from sixty-four to 256 without regularisation drove the gap from 0.1181 to 1.0842, while lowering it to sixteen, alongside a reduced learning rate, removed the capacity to fit at all. Capacity in isolation determines neither outcome; it determines how quickly the model reaches whichever failure the remaining settings permit. Dropout at 0.4 with 128 units cut the gap eightfold and delayed the overfitting turn by eleven epochs without changing a single prediction, resolving the calibration problem while leaving the decision problem untouched. That distinction is invisible to accuracy and F1, and it is the practical reason loss remains the primary diagnostic.

C. Data Volume and Diversity Against Architectural Capacity

The third debrief question asks what happened when the dataset was expanded, and why capacity must be balanced against data volume and representative diversity.

EXP-01 on forty examples scored below chance under the provided defaults, and the same configuration on 225 examples scored 0.9388 macro F1. No hyperparameter change tested here produced an effect comparable in magnitude; the largest tuning-induced improvement among the good runs was 0.0133 of validation loss.

The mechanism is a ratio. EXP-01’s model held roughly nine thousand trainable parameters against twenty-eight training examples. At that ratio, memorising the training set is the path of least resistance, and the resulting weights encode individual sentences rather than transferable patterns. Training accuracy of one hundred percent alongside validation accuracy of 16.67 percent is exactly that condition.

Diversity mattered independently of volume, which extends rather than contradicts the PT-P1 finding. The

expanded corpus was constructed to cover negation, sarcasm, and time-pressure phrasing specifically, and the consequence appears in the cross-evaluation discussed below: T10, the sarcastic phrase no PT-P1 model ever classified correctly, is classified correctly by the champion at 99.78 percent confidence. Targeted coverage resolved in a coded model what ten targeted examples could not resolve in the no-code one.

D. Benchmark Comparison Depends on the Test Set

The champion was evaluated twice on independent data: on the validation split drawn from the expanded corpus, and on the eleven PT-P1 phrases, which were authored before the expanded corpus existed.

TABLE V. CHAMPION EVALUATED AGAINST THREE TEST SETS

Evaluation set	n	Accuracy	F1
Validation split	34	94.12%	0.9388
Held-out test split	34	97.06%	0.9705
PT-P1 fixed test set	11	81.82%	0.8222
PT-P1 no-code benchmark	11	90.91%	0.9153

The first three rows are the same model weights measured on different data; F1 is macro except for the held-out split, which is weighted. The fourth row is the target to be exceeded, measured on the same eleven phrases as the third.

On the validation split the Keras champion exceeds the benchmark, 0.9388 against 0.9153. On the eleven phrases the benchmark was actually measured on, it falls below, 0.8222 against 0.9153, a deficit of 0.0931. The same weights produce opposite conclusions.

The held-out test split, which was never used for tuning, gives 97.06 percent with a single error among thirty-four samples. Its generalisation gap is negative: the model scored 2.94 points higher on test than on validation. A negative gap of this kind normally indicates that the test partition is easier rather than that the model is exceptional, and the explanation here is that the expanded corpus was constructed by paraphrasing a shared set of templates, so a held-out row resembles a training row closely. The figure is reported for completeness, but it is not the generalisation estimate this paper relies on.

The explanation lies in how each set was built. The validation split is drawn from the expanded corpus, whose sentences are paraphrases of shared templates, whereas the PT-P1 set was written independently to probe negation, sarcasm, and the complaint-urgent boundary. Reporting only the validation figure would have supported a claim that the independent evidence contradicts.

TABLE VI. PER-PHRASE CROSS-EVALUATION AGAINST THE NO-CODE MODEL

ID	Case	Exp.	Keras	Conf.	PT-P1
T01	Clear	Pos	Neg	61.36	Pos ✓
T02	Clear	Pos	Pos	100.00	Pos ✓

ID	Case	Exp.	Keras	Conf.	PT-P1
T03	Clear	Pos	Pos	99.98	Pos ✓
T04	Clear	Neg	Neg	100.00	Neg ✓
T05	Negation	Neg	Neg	99.29	Neg ✓
T06	Clear	Neg	Pos	60.99	Neg ✓
T07	Boundary	Urg	Urg	85.87	Urg ✓
T08	Clear	Urg	Urg	99.84	Urg ✓
T09	Clear	Urg	Urg	99.99	Urg ✓
T10	Sarcasm	Neg	Neg	99.78	Urg ✗
T11	Sarcasm	Neg	Neg	99.95	Neg ✓
T12	OOV	—	Neg	97.35	Urg 3%

Keras predictions are from the champion, EXP-05. Confidence in percent. T01 and T06 are the two errors; T12 has no valid class and is excluded from scoring.

The champion resolved T10, which no PT-P1 model ever classified correctly, and lost T01 and T06, both clear cases that the no-code model handled. It traded two easy wins for one difficult one.

Mean confidence on the nine correct predictions is 98.30 percent and on the two errors 61.18 percent, a gap of 37.1 points against PT-P1’s 34.1. The separation is clean: any threshold between roughly sixty-two and eighty-five percent would route both errors to a human agent while passing all nine correct answers, which reproduces and strengthens the confidence-threshold safeguard proposed in PT-P1.

The out-of-vocabulary probe inverts this conclusion. T12 belongs to no trained class. PT-P1 returned three percent confidence, a usable warning, whereas the champion returned Negative_Complaint at 97.35 percent. Softmax constrains the three probabilities to sum to one, so the architecture has no representation for the answer that none of the classes apply and must distribute all probability mass among classes that do not fit. A confidence threshold cannot catch this failure, and it is the most serious weakness identified in the coded model.

V. PIPELINE MAPPING AND AUTOMATED FEATURE EXTRACTION

The fourth debrief question asks where the forward pass, loss calculation, backward pass, and weight update occur in the implemented pipeline, and how the embedding and pooling layers perform feature extraction prior to dense classification.

TABLE VII. LECTURE CONCEPT MAPPED TO NOTEBOOK LOCATION

Stage	Location	Mechanism
Initialisation	Section 3	Glorot uniform, scaled by layer size
Forward pass	Inside model.fit(), Sec. 5	Batch flows to a softmax distribution
Loss calculation	Configured Sec. 4; run in fit	Sparse categorical cross-entropy
Backward pass	Inside model.fit(), Sec. 5	Reverse-mode autodifferentiation

Stage	Location	Mechanism
Weight update	Optimizer Sec. 4; applied in fit	Adam, adaptive per-parameter step
Validation	validation_data, Sec. 5	Forward pass and loss only

Section numbers refer to the submitted notebook. Sparse categorical cross-entropy is correct for integer labels; the categorical variant expects one-hot vectors and would raise a shape error.

All four training steps occur within the single call to model.fit(). They are not separately visible in the source, which is precisely why the loss curve is the instrument for observing them: the per-epoch training and validation losses recorded in the history object are the only externally visible trace of a process that executes seventy-five times per fifteen-epoch run on the expanded corpus.

The distinction between the training and validation paths is worth stating explicitly. The validation data argument triggers a forward pass and a loss computation but no backward pass and no weight update, which is what makes validation loss a generalisation signal rather than a second training score.

Feature extraction is performed by the first two layers. The embedding layer maps each integer token index to a trainable dense vector of thirty-two dimensions. Nothing about word meaning is programmed into it; the vectors initialise randomly and are shaped entirely by gradient descent against the classification loss [9]. Global average pooling then collapses the twenty-by-thirty-two matrix into a single thirty-two-dimensional vector by averaging across token positions, producing a fixed-size representation from a variable-length sentence. Together these replace the manual feature engineering, whether bag-of-words counts, term weighting, or hand-built lexicons, that a classical pipeline would require. Zero-masking excludes padding positions from that average, without which a six-word sentence padded to twenty positions would be diluted by fourteen meaningless slots.

The critical property is that averaging discards word order. The representation reaching the classifier is order-invariant, which is the same limitation PT-P1 attributed to its platform’s apparent bag-of-words representation [5], [7]. The architecture changed; the representational constraint did not. That the champion nevertheless resolved both sarcasm cases indicates the fix operated within the feature space, by giving individual tokens stronger class associations, rather than by extending it to encode relations between them.

VI. LIMITATIONS AND FUTURE WORK

The vocabulary is constructed before partitioning. The vocabulary builder is called on the full dataframe prior to the split, so token frequencies from validation and test rows influence vocabulary membership. This is the same class of issue as fitting a scaler on the full dataset. The leak affects

vocabulary membership only and not labels, and it applies equally to every run, so between-run comparisons remain valid. It is disclosed rather than corrected because correcting it would have invalidated the completed series.

Four PT-P1 test phrases carry high word overlap with expanded-corpus training sentences, ranging from 0.57 to 0.82 by Jaccard similarity on word sets. No exact duplicates exist, but this is an advantage the no-code model did not have, and it renders the 81.82 percent cross-evaluation figure optimistic rather than conservative.

The evaluation sets are small. Thirty-four validation samples and eleven cross-evaluation phrases mean that a single changed prediction moves accuracy by 2.9 and 9.1 percentage points respectively, and no confidence interval on these figures is meaningful. The logging convention is also mixed: runs without early stopping report final-epoch metrics while runs with it report best-epoch metrics, a consequence of best-weight restoration. Finally, all runs use a single seed, so differences below roughly 0.01 in validation loss between good configurations may not survive a change of seed, and the EXP-05 to EXP-05B difference of 0.0075 falls within that range.

Future work would replicate the series across several seeds to establish which differences are stable, rebuild the vocabulary on the training split alone, and introduce an explicit rejection mechanism so that input belonging to no trained class can be identified rather than forced into one. A sequence-aware architecture would test directly whether the order-invariance identified in Section V, and not the quantity of data, sets the ceiling on this task.

VII. CONCLUSION

A Keras feed-forward classifier was trained across seven logged and reproducible configurations and compared against a no-code baseline on the same three-class intent task.

Dataset size was the decisive variable. Holding every hyperparameter, the architecture, and the random seed constant, expanding the corpus from forty to 225 examples moved validation macro F1 from 0.0952 to 0.9388 and ROC-AUC from 0.3333 to 0.9987. Across the four subsequent configurations on the expanded corpus, no change to capacity, dropout, weight decay, or training length altered a single prediction. Tuning had saturated, and only data would move the result further.

Two runs demonstrated that accuracy is insufficient as a diagnostic, and they did so in opposite directions. EXP-02 reported 71.34 percent training accuracy at a chance-level loss, its softmax outputs near-uniform. EXP-03 achieved the highest macro F1 in the series while carrying a validation-training gap of 1.0842 and a best epoch of two of fifteen. Selecting on F1 alone would have shipped the memorising model.

The loss curves carried information that no summary statistic preserved. EXP-03 reached a validation loss of approximately 0.05 at its second epoch, lower than any value the champion attained, and then destroyed that result across thirteen further epochs of oscillation and divergence. Its final metrics record the endpoint of a badly configured training procedure rather than the capability of the model, and only the curve and the best-epoch column make that visible.

The benchmark comparison resolves differently depending on the test set. The champion exceeds the no-code baseline on a random validation split and falls below it on the eleven adversarial phrases the baseline was originally measured against. The honest generalisation estimate is 81.82 percent rather than the 94.12 percent obtainable from same-distribution data.

Finally, the reproducibility defect is worth restating. The two seed lines specified in the provided code did not determine the result, and this was discovered only because a configuration was deliberately re-run to check. A logged seed that does not reproduce is not a record of an experiment; it is a record of an intention.

REFERENCES

- [1] D. Lane, "Machine Learning for Kids," machinelearningforkids.co.uk. [Online]. Available: <https://machinelearningforkids.co.uk>
- [2] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: <https://www.tensorflow.org>
- [3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [5] Z. S. Harris, "Distributional structure," *Word*, vol. 10, no. 2–3, pp. 146–162, 1954.
- [6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- [7] M. Taboada, J. Brooke, M. Tofiloski, K. Voll, and M. Stede, "Lexicon-based methods for sentiment analysis," *Computational Linguistics*, vol. 37, no. 2, pp. 267–307, 2011.
- [8] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 1321–1330.
- [9] Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research*, vol. 57, pp. 345–420, 2016.
- [10] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in Proc. 13th Int. Conf. Artificial Intelligence and Statistics (AISTATS), 2010, pp. 249–256.
- [11] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.